NioLabs

Quantova Bridge Guard Security Assessment

NioLabs security assessment, October 2026

Prepared for Quantova Inc
Reviewed revision, initial public release
Remediation releases v0.2.0 and v0.3.0
Report date 14 October 2026

Executive Summary

Engagement overview

Quantova Inc engaged NioLabs to review the security of Quantova Bridge Guard, a post quantum attestation layer that third party bridges on Ethereum install as an additional verification module. The guard accepts a cross chain message only when a strict majority of a signing committee has attested to a Merkle root containing it with LMS hash based signatures as specified in RFC 8554, optionally paired with secp256k1 ECDSA during migration. The reviewed system comprised an on chain LMS verifier, the guard core, a message security module adapter, a LayerZero V2 decentralized verifier network contract and a Python signer kit that manages one time key state.

The review covered the entire repository at its initial public release on 14 October 2026. NioLabs combined manual review through five lenses with testing on a production toolchain, authoring 99 tests that include 512 run fuzzing, differential testing against an independent RFC 8554 signer and end to end tests that drive the real signer kit.

Summary of findings

The review found no critical and no high severity issues. No path was found to forge a signature, approve a forged message root, deliver an unapproved or altered cross chain message, seize the committee, drain funds or bypass the quorum, and there is no owner or administrator backdoor anywhere in the system. The issues identified are one medium severity assurance gap, nine low severity issues that concern deployment parameters, integration and defence in depth, and seven informational observations.

Severity	Count	Character of the issues
Critical	0	None
High	0	None
Medium	1	No committed test suite at the reviewed revision
Low	9	Deployment parameters, one time key operation, an integration mismatch, defence in depth
Informational	7	An unreferenced constant, a benign memory read, operational rigidity

Quantova Inc addressed every finding in releases v0.2.0 and v0.3.0. Ten findings are resolved through code changes and seven are acknowledged with the rationale recorded in this report. In v0.3.0 Quantova Inc removed the message security module adapter, and bridges now call the guard directly. The remediation status reflects the changes reported by Quantova Inc and is listed with each finding.

Project Summary

Engagement

Item	Detail
Client	Quantova Inc
Assessor	NioLabs
Report date	14 October 2026
Repository	Quantova Bridge Guard, GitHub account quantova inc
Reviewed revision	Initial public release, 14 October 2026
Remediation releases	v0.2.0 and v0.3.0, Quantova Inc
Toolchain	Foundry 1.8.5, solc 0.8.28 with the IR pipeline, Cancun EVM, optimizer runs 10000, Python standard library

Project goals

The engagement sought to answer the following questions.

- Can an LMS signature be forged, or an invalid or malleable signature be accepted, for any supported parameter set?
- Can a root be approved, revoked or a committee rotated without a strict majority of distinct, unfrozen members, including through double counting, replay across actions, contracts, chains or epochs, or reuse of one time key leaves?
- Can an adapter deliver a message that is not included in an approved root, or deliver an altered message, and can adapter funds be diverted?
- Can the signer kit sign one leaf twice under crashes, concurrency, copies, restores or clones of the key state?
- Does the Python kit encode digests, leaves and proofs exactly as the contracts do, so that the kit can never sign something the contracts interpret differently?
- Does the codebase contain any privileged path, upgrade mechanism or hidden control?

Project targets

File	Lines	Role
src/BridgeGuard.sol	392	Quorum, rotation, freezing, replay binding, Merkle inclusion
src/LMSVerifier.sol	134	RFC 8554 LMS verification through the SHA 256 precompile
src/adapters, message module	67	Message security module adapter, removed in v0.3.0
src/adapters/LayerZeroQuantumDVN.sol	125	LayerZero V2 verifier and fee flow
src/interfaces, three files	58	Guard, message module and LayerZero interfaces
kit/bridgeguard/state.py	408	One time key state, high water mark, locking, rebind and retirement
kit/bridgeguard/lms.py	251	RFC 8554 key generation, signing and verification
kit/bridgeguard, five further files	492	Encoding, command line, ECDSA, Keccak and entry points

Project coverage

All 143 functions across the 15 source files were mapped to at least one review lens, and no function was left uncovered. The lenses were the following.

Lens	Scope
LMS verifier cryptography	RFC 8554 and NIST SP 800 208 conformance, forgery and malleability, typecode handling, the verifier and the kit implementations
Guard core	Quorum counting, rotation, possession, freezing, replay and domain binding, access control
Adapters and inclusion	Merkle inclusion soundness, binding of messages to leaves, the message module, the LayerZero verifier and its fund flow
One time key state	Leaf reuse, crash durability, concurrency, host cloning and key generation entropy
Backdoors and parity	Privilege and backdoor sweep, byte level parity between Solidity and Python, fuzzing and coverage accounting

Automated Testing

NioLabs authored 99 tests and ran them on the toolchain listed above. All 99 passed. The suite included the following.

- Verification of valid signatures for every supported parameter set, and verification of signatures produced by an independent RFC 8554 signer written for this review.
- Rejection of 50 structural mutations, 64 cross message forgeries and a 405 case differential corpus.
- A 512 run fuzz campaign showing that every bit of a signature is load bearing, with no malleability and no counterexample.
- Twenty adversarial guard tests with real LMS and ECDSA attestations covering double counting, off by one quorum, frozen and rotated out signatures, leaf reuse and rollback, and replay between approval and revocation and across contracts, chains, epochs and earlier configurations.
- Differential tests through the foreign function interface proving byte for byte agreement between Solidity and Python for the root, revocation and configuration digests, the leaf hash, the Merkle construction, the adapter leaves and the LMS wire format.
- Concurrency tests in which 16 concurrent signers obtained 16 distinct leaves.

Codebase Maturity Evaluation

Category	Rating	Summary
Arithmetic	Strong	Solidity 0.8 checked arithmetic throughout, with no unchecked blocks.
Access controls	Strong	No owner, administrator, proxy, delegatecall, selfdestruct or tx.origin. Trusted addresses are immutable and checked for zero. The permissionless fee functions can only pay the immutable treasury, see finding 8.
Cryptography	Strong	A faithful byte level implementation of single tree RFC 8554 LMS with domain separated Merkle hashing in the style of RFC 6962.
Key management	Moderate	One time key state is durable and serialised on a single host, but clones made below the file system cannot be detected, see the operational note and findings 7 and 10.
Low level manipulation	Satisfactory	Four assembly blocks were reviewed line by line. Each reads memory or writes only to freshly allocated local memory. One benign over read is noted in finding 12.
Decentralization	Satisfactory	Parameters are immutable and there is no privileged role. A committee that freezes its own keys below the threshold halts permanently, see finding 6.
Documentation	Satisfactory	Thorough operator documentation. The documented relayer flow for the message module did not match its module type, see finding 2.
Testing and verification	Weak	No tests were committed at the reviewed revision, see finding 1.

Summary of Findings

ID	Title	Type	Severity	Status
1	No committed test suite	Testing	Medium	Resolved
2	Message module reports a type that carries no metadata	Configuration	Low	Resolved
3	Minimum root age is unvalidated and may be zero	Configuration	Low	Resolved
4	Merkle index check is disabled for proofs of 256 or more nodes	Data validation	Low	Resolved
5	Rotation requires every retained member to attest	Denial of service	Low	Resolved
6	Freezing enough keys halts the guard permanently	Denial of service	Low	Acknowledged
7	One time key counters are unauthenticated	Cryptography	Low	Acknowledged
8	Fee sweep and withdrawal are permissionless	Access controls	Low	Resolved
9	LMS checksum shift holds only for the supported parameter sets	Cryptography	Low	Acknowledged
10	Advisory file locks are unreliable on network file systems	Configuration	Low	Resolved
11	Unreferenced constant LLA	Code quality	Informational	Acknowledged
12	LMS verifier reads past a 34 byte buffer	Code quality	Informational	Resolved
13	Merkle leaf index is not bound into the leaf	Data validation	Informational	Acknowledged
14	assignJob is payable but ignores the value sent	Code quality	Informational	Acknowledged
15	Message module metadata is decoded outside the try block	Error reporting	Informational	Resolved
16	Supported LayerZero destinations cannot be updated	Configuration	Informational	Acknowledged
17	Signer seed is not zeroised in memory	Cryptography	Informational	Acknowledged

Detailed Findings

1. No committed test suite

Severity	Difficulty	Type	Target
Medium	Undetermined	Testing	Repository

Description

The repository contained no tests. The build configuration referenced a test directory and test vectors that were excluded from version control. For a security critical signature verification product, a committed and runnable suite is the main defence against regressions, and the 99 tests written for this review would otherwise not remain with the code.

Exploit scenario

A future change to the quorum logic or the encoding silently breaks a security property and is released, because no regression suite runs before release.

Recommendations

Commit a test suite seeded with the properties established in this review, including the 512 run fuzzing and the Solidity and Python parity tests, and run it in continuous integration on every change.

Remediation status

Resolved. The repository now ships Foundry tests including fuzzing, RFC 8554 vectors, 24 rounds of randomised Solidity and Python parity and a gas benchmark suite, together with the RFC vector and safety suites of the signer kit and a live smoke test on a local Ethereum node. Continuous integration runs all of them on every push.

2. Message module reports a type that carries no metadata

Severity	Difficulty	Type	Target
Low	Low	Configuration	src/adapters, message security module

Description

The message security module adapter returned the NULL module type, which instructs relayers to build no metadata. The documentation stated that the relayer passes the root, index and proof as metadata. Delivery still required genuine inclusion, so this is a liveness and integration issue rather than a bypass, but deliveries performed as documented would fail.

Exploit scenario

A bridge configures the module and its relayer delivers messages with empty metadata. Every message that was not attested in advance is rejected and the route stalls.

Recommendations

Return the module type that carries metadata to the module.

Remediation status

Resolved in v0.2.0 by moving the module to the offchain lookup type with a proof service in the signer kit. In v0.3.0 Quantova Inc removed the adapter, and bridges now call isIncluded on the guard directly.

3. Minimum root age is unvalidated and may be zero

Severity	Difficulty	Type	Target
Low	High	Configuration	src/BridgeGuard.sol constructor and isApproved

Description

The optimistic revocation window could be configured to zero, so an approved root became usable in the same block. Finality of approval relies on block.timestamp.

Exploit scenario

An operator deploys with a zero window. A committee that approves a wrong root has no opportunity to revoke it before messages under that root are delivered.

Recommendations

Enforce a sensible minimum and document how operators should choose the window.

Remediation status

Resolved in v0.2.0. The constructor rejects any minimum root age below 60 seconds, and the documentation explains the choice of window.

4. Merkle index check is disabled for proofs of 256 or more nodes

Severity	Difficulty	Type	Target
Low	High	Data validation	src/BridgeGuard.sol computeRoot

Description

The check that the leaf index fits the proof depth shifted the index by the proof length. For a proof of 256 or more nodes the shift yields zero and the check no longer applies. Exploitation would still require a Keccak collision, and fuzzing confirmed the behaviour.

Exploit scenario

None practical. The issue weakens a defensive check rather than enabling an attack.

Recommendations

Bound the proof length explicitly.

Remediation status

Resolved in v0.2.0. computeRoot returns zero for any proof longer than 64 nodes or any index that does not fit the proof depth.

5. Rotation requires every retained member to attest

Severity	Difficulty	Type	Target
Low	Medium	Denial of service	src/BridgeGuard.sol reconfigure

Description

Each member of a new configuration had to be proven either through a recorded proof of possession or through its own attestation in the rotating quorum. An honest member that was offline could therefore not be retained in a rotation.

Exploit scenario

A committee needs to rotate urgently while one honest member is unreachable. The member must be dropped and later re added through a new proof of possession.

Recommendations

Consider accepting retained members without a fresh signature while keeping the protection against rogue keys for new members.

Remediation status

Resolved in v0.2.0. A member whose LMS key is in the current committee is accepted without a fresh signature. In hybrid mode its ECDSA address must also be unchanged. New members, and members whose classical key changes or who move from post quantum to hybrid mode, must still prove possession.

6. Freezing enough keys halts the guard permanently

Severity	Difficulty	Type	Target
Low	High	Denial of service	src/BridgeGuard.sol freezing

Description

If enough keys are frozen that fewer unfrozen members remain than the threshold, both approval and rotation become impossible and the parameters are immutable. Freezing requires equivocation by the key holder, which cannot be forged against honest keys, so the condition can only be reached by insiders acting against themselves.

Exploit scenario

A majority of members equivocates, every one of their keys is frozen and the guard can no longer approve or rotate.

Recommendations

Consider a recovery path.

Remediation status

Acknowledged. Halting is the intended fail safe outcome when a majority of keys has misbehaved, because any recovery path that bypasses the committee would itself be a privileged path. v0.2.0 adds liveSignerCount so operators can monitor the margin, and documents recovery by deploying a new guard that the bridge selects through its own configuration.

7. One time key counters are unauthenticated

Severity	Difficulty	Type	Target
Low	High	Cryptography	kit/bridgeguard/state.py

Description

The leaf counters are stored as plain JSON. Rolling back the local state, the journal and the high water mark together resumes signing at a lower index and yields two valid signatures at one leaf. This requires write access to the key directory, which already holds the seed, so such an actor could forge directly and gains nothing.

Exploit scenario

An actor with write access to the key directory and the high water mark directory rolls all three counters back.

Recommendations

Consider a message authentication code over the counter state.

Remediation status

Acknowledged. A message authentication code keyed from material on the same host does not resist an actor who can read the seed. The high water mark must live on storage separate from the key, the kit refuses network file systems, and a hardware module with a monotonic counter is recommended for production custody.

8. Fee sweep and withdrawal are permissionless

Severity	Difficulty	Type	Target
Low	Low	Access controls	src/adapters/LayerZeroQuantumDVN.sol

Description

sweep and withdrawFromLibrary could be called by anyone. No theft is possible because the treasury is immutable and checked for zero, and the transfer is safe against reentrancy.

Exploit scenario

A third party triggers withdrawals at times the operator did not choose.

Recommendations

Restrict both functions for hygiene.

Remediation status

Resolved in v0.2.0. Only the treasury can call sweep and withdrawFromLibrary.

9. LMS checksum shift holds only for the supported parameter sets

Severity	Difficulty	Type	Target
Low	High	Cryptography	src/LMSVerifier.sol

Description

The checksum left shift matches the RFC 8554 definition and the kit for every shipped parameter set. It would need to be recomputed if a wider parameter set were added.

Exploit scenario

A future maintainer adds a parameter set without updating the shift, and the verifier computes an incorrect checksum.

Recommendations

Re check the shift whenever a parameter set is added.

Remediation status

Acknowledged. The parameter table admits only the four RFC 8554 LMOTS types with their defined shifts, and any other typecode is rejected.

10. Advisory file locks are unreliable on network file systems

Severity	Difficulty	Type	Target
Low	Medium	Configuration	kit/bridgeguard/state.py

Description

The kit serialises signers with flock, which is advisory and unreliable over NFS. This matters only outside the documented model of one host and local storage.

Exploit scenario

A key directory is placed on a shared network volume and two hosts sign concurrently, reusing leaves.

Recommendations

Document the constraint prominently.

Remediation status

Resolved in v0.2.0. The kit inspects the mount table on Linux and macOS and refuses key and high water mark directories on network file systems such as NFS, SMB, AFP, WebDAV, SSHFS, Ceph and GlusterFS. The constraint is documented.

11. Unreferenced constant LLA

Severity	Difficulty	Type	Target
Informational	Undetermined	Code quality	src/BridgeGuard.sol

Description

The constant LLA holding 16121994 is never referenced and has no effect. An unexplained value in a security contract invites questions.

Recommendations

Remove the constant or explain it.

Remediation status

Acknowledged. The constant is retained by design as the Quantova Inc authorship stamp in the deployed bytecode. It is read only, takes no part in verification, and is now documented.

12. LMS verifier reads past a 34 byte buffer

Severity	Difficulty	Type	Target
Informational	Undetermined	Code quality	src/LMSVerifier.sol

Description

Digit extraction loads a 32 byte word near the end of the 34 byte buffer holding Q and its checksum, reading up to 31 bytes beyond it, and uses only the top byte. The behaviour is correct and memory safe.

Recommendations

Pad the buffer.

Remediation status

Resolved in v0.2.0. The buffer is padded to 64 bytes so every load stays within it.

13. Merkle leaf index is not bound into the leaf

Severity	Difficulty	Type	Target
Informational	Undetermined	Data validation	src/BridgeGuard.sol isIncluded

Description

The index only selects the direction at each level of the path. The adapters key their state on the message identifier and payload, not on the index.

Recommendations

None required.

Remediation status

Acknowledged. Every leaf binds the full message, so the index carries no meaning beyond the path.

14. assignJob is payable but ignores the value sent

Severity	Difficulty	Type	Target
Informational	Undetermined	Code quality	src/adapters/LayerZeroQuantumDVN.sol

Description

The function is payable to match the LayerZero interface and ignores msg.value. Only the send library can call it.

Recommendations

None required.

Remediation status

Acknowledged. The signature is fixed by the LayerZero interface, and any value received can only be swept to the treasury.

15. Message module metadata is decoded outside the try block

Severity	Difficulty	Type	Target
Informational	Undetermined	Error reporting	src/adapters, message security module

Description

Malformed metadata reverted the delivery attempt instead of returning false. It never made verification return true.

Recommendations

Decode inside the guarded call.

Remediation status

Resolved in v0.2.0 by decoding inside the guarded call. The adapter was removed in v0.3.0.

16. Supported LayerZero destinations cannot be updated

Severity	Difficulty	Type	Target
Informational	Undetermined	Configuration	src/adapters/LayerZeroQuantumDVN.sol

Description

The set of supported destination endpoints is fixed at construction.

Recommendations

None required.

Remediation status

Acknowledged. Immutability is intended because it removes any privileged role. New destinations are served by a new deployment.

17. Signer seed is not zeroised in memory

Severity	Difficulty	Type	Target
Informational	Undetermined	Cryptography	kit/bridgeguard/state.py

Description

The seed must persist in the key file with owner only permissions and cannot be reliably zeroised in CPython memory. No secret is written to logs.

Recommendations

None required.

Remediation status

Acknowledged. This is a limitation of the runtime. Hardware custody is recommended for production keys.

Operational Note on Host Cloning

LMS is a one time signature scheme in which reusing a leaf exposes key material and enables forgery. The signer kit is carefully built. It persists the index durably before releasing a signature, serialises signers with a lock, and detects copies made with cp, rsync or a restore. Its copy detector, however, relies on the directory inode, which a block level clone, a virtual machine snapshot or a disk image preserves. Two live clones would share the fingerprint and could reuse leaves. NioLabs considers this the risk most likely to matter in production despite the clean on chain result. Operators should run one signer per key on local storage that cannot be snapshotted, or keep the counter in a hardware module with a monotonic counter.

Quantova Inc reports that v0.2.0 documents this constraint prominently, keeps the high water mark on separate storage and refuses network file systems.

Recommendations

- Commit a test suite and continuous integration, including the 512 run fuzzing and the Solidity and Python parity tests.
- Correct the message module type so documented relayer deliveries succeed.
- Publish operational guidance for the signer on host cloning, hardware counters and network file systems.
- Validate the minimum root age with a sensible floor and document the revocation window.
- Explain or remove the unreferenced constant, restrict the fee functions, and consider authenticated counters and a recovery path.

Notices

This report reflects the state of the repository at the reviewed revision and the findings of a time bounded review. It is not a guarantee that the system is free of defects. Remediation statuses describe the changes reported by Quantova Inc in releases v0.2.0 and v0.3.0. Integrators remain responsible for reviewing and testing their own deployments.